Before we start

- **Logistics**
  - Check in
  - Download the latest materials, especially class activities
- **Staying on track**
  - Mind the deadlines
  - Follow instructions for submissions
  - Don't fall behind: the course is short, and ramps up in complexity quickly
- **Preparation and support**
  - Come prepared: review the activity and slides beforehand
  - Get in touch with questions and concerns



Data Structures and Algorithms

Functions and Algorithms

Prof. Reza Skandari

Today

1. **Recap**
2. Functions
3. A first algorithm

Test your instincts

Poll time! Open menti.com and vote on what you think each snippet does.

What is the output?

```
# What is the output?
x = 5
6 = y
print(x)
print(y)
```

A. 5, then 6

B. 5, then 5

C. 6, then 6

D. An error

E. I don't know

What is the output?

```
## What is the output?
a = 2
b = a
a = 5

print(a)
print(b)
```

A. 2, then 2

B. 2, then 5

C. 5, then 2

D. 5, then 5

E. I don't know

What is the output?

```
# What is the output?
x = 5
if x >= 0:
    print(1)
elif x < 20:
    print(2)
else:
    print(3)
print(4)
```

A. 1, then 2, then 4

B. 1, then 4

C. 4

D. 3, then 4

E. I don't know

What is the output?

```
# What is the output?
x = 3
while x > 0:
    print(x)
    x = x - 1
```

A. 3, 2, 1

B. 3, 2, 1, 0

C. 3, 2

D. 2, 1, 0

E. I don't know

What is the output?

```
# What is the output?
x = 3
while x > 0:
    x = x - 1
print(x)
```

A. 3, 2, 1

B. 2, 1, 0

C. 1

D. 0

E. I don't know

Today

1. Recap
2. **Functions**
3. A first algorithm

We have already been using functions

In Session 1, we used built-in functions:

```
>>> abs(-3)
3
>>> max(5, 3, 10)
10
>>> max(abs(-5), min(3, 9))
```

We say we **call** the function, specifying the arguments within parentheses.

What happens when we do this, and why are functions useful?

We use functions to organise tasks

A function is a named group of statements to perform a specific task.

- Input data → function → output data

```
# Let's define a function abs_value
def abs_value(a):
    if a < 0:
        return -a  # stops execution, outputs -a
    else:
        return a

y = abs_value(-3)  # runs abs_value for a = -3, assigns result to y
```

A function is like a **factory**: in goes input data (car parts), out comes output data (car).

It may have multiple parameters, and return multiple values, separated by commas.

Why functions?



1. **Abstraction**: Hides internal details so users can focus on what the code does, not how it works.
2. **Reusability and modularity**: Code is organized into reusable parts, reducing repetition and improving structure.
3. **Easier maintenance**: Changes can be made in one place without affecting multiple parts of the program.

Today

1. Recap
2. Functions
3. **A first algorithm**

Solving computational problems

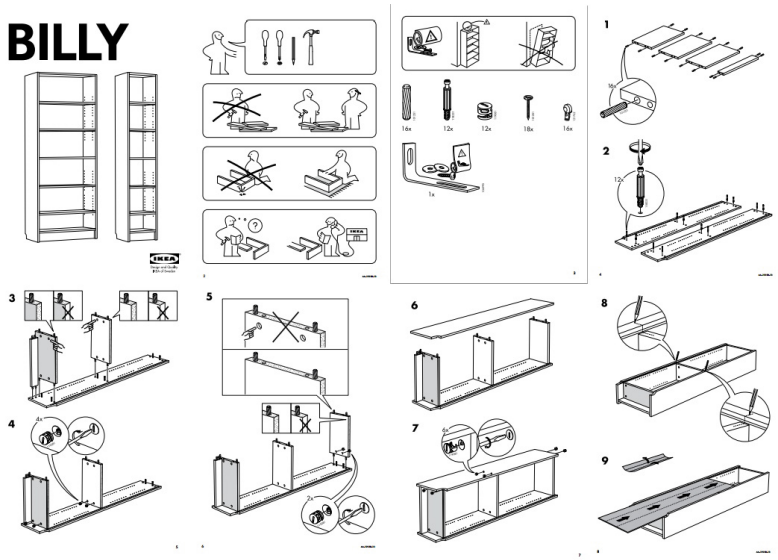
Data = digitised information

> **Data structures** describe ways to organise data
>
> **Algorithms** describe how we process data:
>
> - Step-by-step instructions
> - Take input data and produce output data
>
> We write algorithms into **programs** (eg in Python)

Computers interpret and execute programs

An algorithm is a recipe





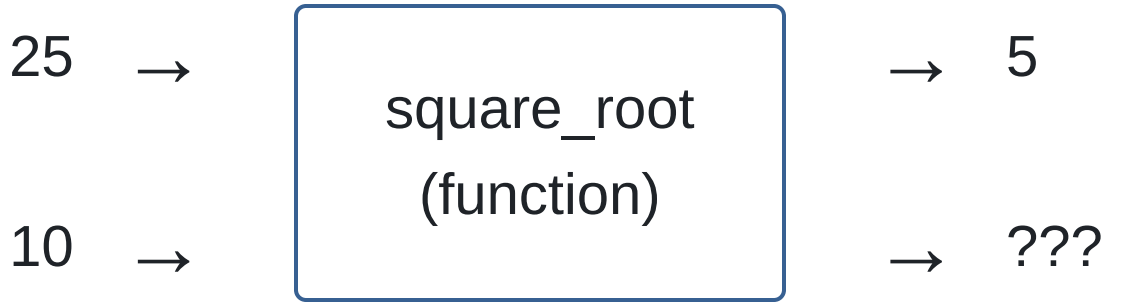
Algorithm:

- Step-by-step instructions
- Takes input (data) and produces output (data)

Pics: Hungry Gals, IKEA.

How do you calculate a square root?

The square root of x is y such that $y \times y = x$



A function is like a factory

- In goes number, out comes square root
- Inside the factory, there's an **algorithm**

Square-root algorithm

The square root of x is y such that $y \times y = x$

Algorithm (Heron of Alexandria, first century AD):

- Make a guess, for example $x/2$
- Repeat three times:
  - Divide the original number x by the guess to get a ratio
  - Find the average of the guess and the ratio
  - Use this average as the next guess

Let's use Python

Square-root function

```
def square_root(x):
    guess = x/2
    eps = 0.01
    while abs(guess*guess-x) >= eps:
        guess = (guess + x/guess)/2
    return guess

z = 20
y = square_root(z)
```

- Takes input *x* and outputs its square root
- Note: uses another function inside it: built-in function `abs`
- Abstraction, reusability, reliability

Today

1. Recap
2. Functions
3. A first algorithm

Review

Algorithms are recipes for solving problems

We divide programs into named functions:

- Reusable code
- User can "just use" the function

Up next:

- Session 2: Exercises on working with functions
- Session 3: Working with for loops and lists